CSE2331 Final Exam Review 1

This assignment will be graded entirely based on **participation**.

Problem 1. For the following algorithm,

(i) Determine the best case running time and state what condition must be true for the best case running time occur.

Use $B(n)$ to denote the best case running time of the algorithm with an input of size n.

(ii) Determine the worst case running time and state what condition must be true for the worst case running time occur.

Use $W(n)$ to denote the worst case running time of the algorithm with an input of size n.

(iii) Determine the expected case running time.

Use $ET(n)$ to denote the expected running time of the algorithm with an input of size n.

Assume that Random(n) generates a random integer between 1 and n with uniform distribution.

```
function Func(n)
    x = 0
    k = Random(n)
    i = k^2
    while i ≥ k do
        for j = 1 to k do
            x = x + i − j
        end for
        i = i − 2
    end while
    return x
end function
```

$\Theta(k^2)$

Itr	i
0	k^2
1	k^2-2
⋮	
ℓ	$k^2-2\ell$

Stops when

$k^2-2\ell = k$

$\ell = \frac{k^2-k}{2} \approx k^2/2$

a) B.C. occurs when $k=1$

$B(n) \in \Theta(1)$

b) W.C. occurs when $k=n$

$W(n) \in \Theta(n^3)$.

c) $ET(n) = \sum_{q=1}^{n} Pr(k=q)\, t(k=q)$

$= \sum_{q=1}^{n} \frac{1}{n} \cdot cq^3 = \frac{c}{n} \sum_{q=1}^{n} q^3 = \frac{c}{n} \cdot \frac{n^2(n+1)^2}{4}$

Thus,

$ET(n) \in \Theta(n^3)$.

Problem 2. For the following algorithm

(i) Determine the best case running time and state what condition must be true for the best case running time occur.

Use $B(n)$ to denote the best case running time of the algorithm with an input of size n.

(ii) Determine the worst case running time and state what condition must be true for the worst case running time occur.

Use $W(n)$ to denote the worst case running time of the algorithm with an input of size n.

(iii) Determine the expected case running time.

Use $ET(n)$ to denote the expected running time of the algorithm with an input of size n.

Assume that $\texttt{Random}(n)$ generates a random integer between 1 and n with uniform distribution.

```
function Func(A[],n)
    if n ≤ 1 then
        return 1
    end if
    x = 0
    for i = 1 to n^2 do
        x = x + A[⌊i/n⌋]
    end for
    k = Random(n)
    if k ≤ n/4 then
        x = x + Func(A[], n − 3)
    else
        x = x + Func(A[], n/4)
    end if
    return x
end function
```

a) B.C. occurs when $k > \frac{n}{4}$ every time

$B(n) = cn^2 + B(n/4)$, thus $B(n) \in \Theta(n^2)$.

b) W.C. occurs when $k \le n/4$ always

$W(n) = cn^2 + W(n-3)$, thus $W(n) \in \Theta(n^3)$.

c) $ET(n) = cn^2 + \frac{1}{4}ET(n-3) + \frac{3}{4}ET(n/4)$

$ET(n) \le cn^2 + \frac{1}{4}ET(n) + \frac{3}{4}ET(n/4)$

$\frac{3}{4}ET(n) \le cn^2 + \frac{3}{4}ET(n/4)$

$ET(n) \le \frac{4}{3}cn^2 + ET(n/4)$

$ET(n) \le \alpha n^2 + ET(n/4)$

Assume that $\texttt{Random}(n)$ returns values from 1 to n (including both endpoints) uniformly.

$ET(n) \le \alpha n^2 + \alpha(n/4)^2 + ET(n/16)$

$\le \alpha n^2 + \alpha(n/4)^2 + \alpha(n/16)^2 + ET(n/64)$

$ET(n) \le \sum_{i=0}^{\gamma-1} \alpha\left(\frac{n}{4^i}\right)^2 + ET(n/4^\gamma)$

Choose γ s.t. $\frac{n}{4^\gamma} = 1$, $\gamma = \log_4(n)$

$\le \sum_{i=0}^{\infty} \frac{\alpha n^2}{16^i} + c$

$\le \alpha n^2 \left[\frac{1}{1 - 1/16}\right]$

$ET(n) \in O(n^2)$

The BC is a lower bound so $ET(n) \in \Omega(n^2)$.

Thus, $ET(n) \in \Theta(n^2)$.

Problem 3. For the following algorithm,

(i) Determine the best case running time and state what condition must be true for the best case running time occur.

Use $B(n)$ to denote the best case running time of the algorithm with an input of size n.

(ii) Determine the worst case running time and state what condition must be true for the worst case running time occur.

Use $W(n)$ to denote the worst case running time of the algorithm with an input of size n.

(iii) Determine the expected case running time.

Use $ET(n)$ to denote the expected running time of the algorithm with an input of size n.

Assume that $\texttt{Random}(n)$ generates a random integer between 1 and n with uniform distribution.

(a)

```
function Func(A[],n)
    if n ≤ 2 then
        return A[1]
    end if
    for i = 1 to n do
        for j = 1 to ⌊n/2⌋ do
            for m = 1 to 2⌊√n⌋ do
                A[1] = A[1] + A[i] · A[j] − A[m]
            end for
        end for
    end for
    k = Random(n/2)
    A[1] = A[1] + Func(A[], n − k)
    return A[1]
end function
```

$$T(n) = cn^{5/2} + T(n-k)$$

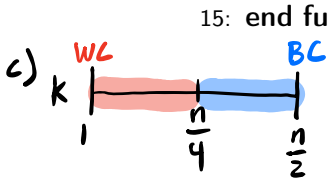
a) BC occurs when $k = n/2$ always.

$B(n) = cn^{5/2} + B(n/2)$. So, $B(n) \in \Theta(n^{5/2})$.

b) WC occurs when $k = 1$ always

$W(n) = cn^{5/2} + W(n-1)$. So, $W(n) \in \Theta(n^{7/2})$.

c)



$$ET(n) = E(T(n) \mid k \le n/4)\Pr(k \le n/4) + E(T(n) \mid k > n/4)\Pr(k > n/4)$$

$$ET(n) \le [cn^{5/2} + ET(n-1)](1/2) + [cn^{5/2} + ET(3n/4)](\tfrac{1}{2})$$

$$ET(n) \le cn^{5/2} + \tfrac{1}{2}ET(n) + \tfrac{1}{2}ET(3n/4)$$

$$\tfrac{1}{2}ET(n) \le cn^{5/2} + \tfrac{1}{2}ET(3n/4)$$

$$ET(n) \le 2cn^{5/2} + ET(3n/4)$$

$$ET(n) \le \alpha n^{5/2} + ET(3n/4)$$

$$ET(n) \le \alpha n^{5/2} + \alpha(3n/4)^{5/2} + ET(9n/16)$$

$$ET(n) \le \alpha n^{5/2} + \alpha(3n/4)^{5/2} + \alpha(9n/16)^{5/2} + ET(27n/64)$$

$$ET(n) \le \sum_{i=0}^{\gamma-1} \alpha\left(\frac{3^i n}{4^i}\right)^{5/2} + ET\left(\frac{3^\gamma n}{4^\gamma}\right)$$

choose γ s.t. $\frac{3^\gamma n}{4^\gamma} = 1$

$\gamma = \log_{4/3}(n)$

$$\le \alpha n^{5/2} \sum_{i=0}^{\infty} \left(\frac{3^{5/2}}{4^{5/2}}\right)^i + c$$

$$\le \alpha n^{5/2}\left[\frac{1}{1 - 3^{5/2}/4^{5/2}}\right] + c$$

Thus, $ET(n) \in O(n^{5/2})$.

The BC is a lower bound, so $ET(n) \in \Omega(n^{5/2})$.

So, $ET(n) \in \Omega(n^{5/2})$.

(b)

```
function Func(n)
    x = 0
    i = 1
    for i = n to n^2 do
        for j = 1 to n^3 do
            x = x + i
        end for
        k = Random(n)
        if k mod 8 ≥ 5 then
            return x
        end if
    end for
    return x
end function
```

Let $X = \#$ of times $k \bmod 8 < 5$

$T(n) = cn^3 + Xcn^3$

a) BC occurs when $k \bmod 8 < 5$ the first time.

$B(n) \in \Theta(n^3)$.

b) WC occurs when $k \bmod 8 \geq 5$ always.

$(n) \in \Theta(n^5)$.

c) $ET(n) = cn^3 + cn^3 E(X)$

$$= cn^3 + cn^3 \sum_{i=1}^{n^2-n} Pr(X \geq i)$$

$$= cn^3 + cn^3 \sum_{i=1}^{n^2-n} \left(\tfrac{5}{8}\right)^i$$

$$\left.\begin{aligned} &\leq cn^3 + cn^3 \sum_{i=0}^{\infty} \left(\tfrac{5}{8}\right)^i \\ &\leq cn^3 + cn^3 \left[\frac{1}{1-5/8}\right] \end{aligned}\right\} ET(n) \in O(n^3).$$

$$\geq cn^3 \quad \Big\} \; ET(n) \in \Omega(n^3).$$

Thus, $ET(n) \in \Theta(n^3)$.

Problem 4. For the following algorithm

(i) Determine the best case running time and state what condition must be true for the best case running time occur.

Use $B(n)$ to denote the best case running time of the algorithm with an input of size n.

(ii) Determine the worst case running time and state what condition must be true for the worst case running time occur.

Use $W(n)$ to denote the worst case running time of the algorithm with an input of size n.

(iii) Determine the expected case running time.

Use $ET(n)$ to denote the expected running time of the algorithm with an input of size n.

```
function Func(A[],i,j)
    x = 0
    m = Random(i, j)
    for k = i to m do
        h = m
        while h ≥ i do
            x = x + A[h] + A[k]
            h = h − 2
        end while
    end for
    return x
end function
```

$c(m-i)$

$c(m-i)(m-i+1) \approx c(m-i)^2$

Iteration	h
0	m
1	m-2
2	m-4
⋮	⋮
γ	$m-2\gamma$

Stops when

$m - 2\gamma = i$

$\gamma = \frac{m-i}{2}$

Assume that $\texttt{Random}(i, j)$ returns values from i to j (including both endpoints) uniformly.
Let $n = j - i + 1$

i) BC occurs when $m = i$.

$B(n) \in \Theta(1)$

ii) WC occurs when $m = j$.

$W(n) \in \Theta(n^2)$

iii) $ET(n) = \sum_{q=i}^{j} c(q-i)^2 \cdot \cancel{\frac{1}{j-i+1}}^{\,1/n} = \frac{c}{n}\left[0 + 1^2 + 2^2 + \dots (j-i)^2\right]$

$= \frac{c}{n} \sum_{x=1}^{n-1} x^2$

$= \frac{c}{n}\left(\frac{(n-1)n(2n-1)}{6}\right)$

Thus,

$ET(n) \in \Theta(n^2)$.

Problem 5. For the following algorithm

(i) Determine the best case running time and state what condition must be true for the best case running time occur.

Use $B(n)$ to denote the best case running time of the algorithm with an input of size n.

(ii) Determine the worst case running time and state what condition must be true for the worst case running time occur.

Use $W(n)$ to denote the worst case running time of the algorithm with an input of size n.

(iii) Determine the expected case running time.

Use $ET(n)$ to denote the expected running time of the algorithm with an input of size n.

```
function Func(A[],n)
    x = 0
    for k = 1 to n^2 do
        x = x + Foo(A[], k+1, n^2)
    end for
    return x
end function
```

$\rightarrow$ size $= n^2 - (k+1) + 1 = n^2 - k$

Let $n = j - i + 1$ and assume that

- The best case running time of $\texttt{Foo}(A[], i, j)$ is $\Theta(\log(n))$.
- The expected case running time of $\texttt{Foo}(A[], i, j)$ is $\Theta(\sqrt{n})$.
- The worst case running time of $\texttt{Foo}(A[], i, j)$ is $\Theta(n)$.

i) BC occurs when Foo always yields its BC.

$$B(n) = \sum_{k=1}^{n^2} c\log(n^2-k)$$

$$\leq \sum_{k=1}^{n^2} c\log(n^2) \qquad \geq \sum_{k=1}^{n^2/2} c\log\left(n^2-\frac{n^2}{2}\right)$$

$$\leq n^2 \cdot c\log(n^2) \qquad \geq \frac{n^2}{2} c\log(n^2/2)$$

Thus, $B(n) \in \Theta(n^2 \log(n))$

ii) WC occurs when Foo always yields its WC.

$$W(n) = \sum_{k=1}^{n^2} c(n^2 - k) = \sum_{k=1}^{n^2} cn^2 - c\sum_{k=1}^{n^2} k$$

$$= n^2 \cdot cn^2 - c\,\frac{n^2(n^2+1)}{2}$$

$$= \frac{cn^4}{2} + \cdots$$

Thus, $W(n) \in \Theta(n^4)$.

iii) $$ET(n) = \sum_{k=1}^{n^2} c\sqrt{n^2 - k}$$

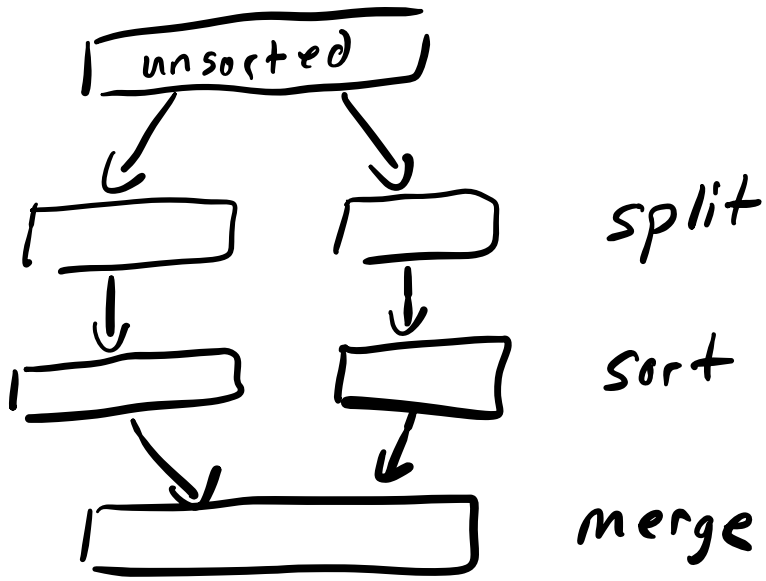
$$\leq \sum_{k=1}^{n^2} c\sqrt{n^2} \qquad \geq \sum_{k=1}^{n^2/2} c\sqrt{n^2 - n^2/2}$$

$$\leq n^2 \cdot cn \qquad \geq \frac{n^2}{2} \cdot \frac{cn}{\sqrt{2}}$$

Thus, $ET(n) \in \Theta(n^3)$.

Problem 6. Explain as much as you can about ALL of the sorting algorithms we discussed in class.

Merge Sort $\Theta(n\log(n))$



Selection Sort

Find the max and place it at the start, repeat recursively.

$\Theta(n^2)$. SLOW!

Quick Sort

Partition around a pivot and sort recursively.

$\leq p$	p	$> p$

BC / EC: $\Theta(n\log(n))$

WC: $\Theta(n^2)$

How to pick a pivot

1) Random
2) First element / arbitrary.
3) Median
4) Average
5) Average of 3
6) Median of 3

Insertion Sort

Iteratively take one element from the unsorted part and correctly insert it into the sorted part.

sorted	unsort

WC: $\Theta(n^2)$

BC: $\Theta(n)$

EC: $\Theta(n^2)$

Heap Sort

Insert all elements into a heap then repeatedly extractMax and place at the end of the array.

$\Theta(n\log(n))$

Generically good.

No meaningful downside

Slightly slower on average than QS.